Exercise 7: Template Renderer

Goal

Build a /render endpoint that accepts two query parameters — title and body — and renders them into an inline HTML template. The template must be defined inside your Go file as a string constant, parsed with html/template, and executed into the ResponseWriter. No external HTML files.

Key Tasks

- Define an HTML template as a raw string constant inside your .go file:

```
const tmplStr = `
<!DOCTYPE html>
<html>
<head><title>{{.Title}}</title></head>
<body>
  <h1>{{.Title}}</h1>
  <p>{{.Body}}</p>
</body>
</html>
`

type PageData struct {
      Title string
      Body  string
}
```

- Parse the template using template.Must(template.New("page").Parse(tmplStr)).
- In the /render handler — read title and body from the query string.
- If either is missing — return 400 with: "title and body are required".
- Execute the template with tmpl.Execute(w, PageData{Title: title, Body: body}).
- If Execute returns an error — return 500 with: "template execution failed".
- Set the Content-Type header to text/html before executing the template.

Critical difference from Exercise 2 —

In Exercise 2 you set Content-Type before writing the body. Here you must also set it before calling tmpl.Execute() — because Execute writes to w directly. Once Execute writes its first byte, headers are locked. Set them first.

What is template.Must() — and when does it panic? Write the answer in a comment.

Stretch — do this after the core task works

- Add a third query parameter: style. If style=bold, wrap {{.Body}} in tags.
  - Hint: you cannot use an if statement in the query param — use template conditionals: {{if eq .Style \"bold\"}}{{.Body}}{{else}}{{.Body}}{{end}}
- Add a PageData field Style string and pass it through from the query param.